

SCANNING
THE MATRIX

Scanning the Matrix: Harnessing Automation for Vulnerability Scans

UTD Team 3

Technical Directors

- Bre'Ona Williams
- Jerome Lomax
- Regis Wilson

Mentor

- Brian Ricks

Members

- Nishanth Badim
- Harika Gattu
- Divya-Bala-Sri Gogineni
- Bhavana-Bramaram Gurram
- Savyasachi Kokkiralala

Agenda

Problem Statement

Project Objectives/Goals

Investigator Biographies

Research Plan Summary

Project Deliverables

Timeline

Conclusions to date

Issues/Challenges

Bibliography

Acronyms

QnA

PROBLEM STATEMENT

- Conduct a comprehensive analysis of various SAST and DAST tools, comparing automated and manual approaches across multiple factors. The research will evaluate the effectiveness of these tools, with a focus on AI integration, to provide a data-driven conclusion on which method offers superior performance.



PROJECT OBJECTIVES/GOALS

1

Evaluate
Automated
Vulnerability
Analysis Tools

2

Analyze AI
Integration in
Vulnerability
Assessments

3

Develop a Hybrid
Approach to
Security Testing

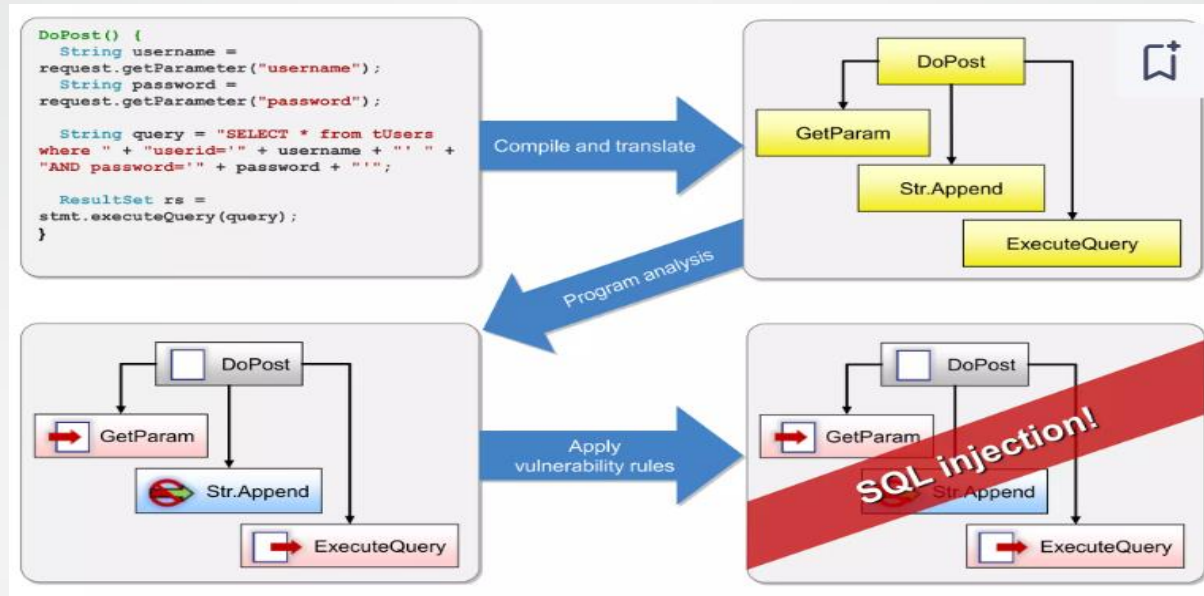
4

Measure Success
with Defined
Metrics

INVESTIGATOR BIOGRAPHIES

- Nishanth Badim will lead the evaluation and benchmarking of automated vulnerability analysis tools using real-world applications. He will be responsible for coordinating the comparison between traditional SAST/DAST tools and AI-driven solutions. Nishanth will also collaborate with Bhavana to ensure that the benchmarking results are integrated into the final project report.
- Harika Gattu will focus on researching AI integration in vulnerability assessments, analyzing AI's potential for improving vulnerability detection, and proposing new AI-driven approaches. She will ensure that all AI implementations adhere to security best practices within the Software Development Life Cycle (SDLC) and work closely with Savyasachi on securely integrating AI into the process.
- Divya-Bala-Sri Gogineni will conduct case studies on real-world cloud and web-based applications to test the assessment methodologies. She will document findings from both manual and automated assessments and assist with validating AI integration into the SDLC by identifying potential risks. Additionally, she will collaborate with Bhavana on compiling these findings for the project report.
- Bhavana-Bramaram Gurram will oversee the comparison between manual and automated vulnerability assessments. She will evaluate the limitations of manual methods and identify areas where AI can complement traditional assessments. Bhavana will also coordinate the writing and review of the project report, ensuring that inputs from all team members are integrated and the findings are clearly documented and aligned with the project's objectives.
- Savyasachi Kokkerala will lead the research on securely integrating AI into the SDLC. He will develop improved vulnerability assessment methodologies by combining AI and traditional techniques. Savyasachi will ensure that all findings align with industry best practices and security standards, and he will collaborate with Harika to ensure that AI practices are both secure and ethical throughout the project.

RESEARCH PLAN SUMMARY



Static Application Security Testing (SAST)

- SAST is a methodology that analyzes source code or compiled applications for security vulnerabilities without executing the application.
- SAST tools allow developers to detect vulnerabilities early in the SDLC by scanning source code, bytecode, or binaries for weaknesses such as SQL Injection, Cross-Site Scripting (XSS), and buffer overflows.
- Upon identifying a vulnerability, a SAST solution generates a detailed report with code-level insights, helping developers quickly locate and fix issues before the code is deployed to production environments. These tools provide continuous scanning during development to prevent security flaws.

SonarQube

1. Continuous Code Quality Analysis
2. Bug Detection and Code Smell Identification
3. Vulnerability Detection
4. Supports Multiple Languages
5. Integration with CI/CD Pipelines

ShiftLeft

1. Real-Time Code Analysis
2. Embedded Security Checks in Development Pipeline
3. Early Vulnerability Detection
4. Developer-Centric Security Insights
5. Continuous Feedback for Code Fixes



Code QL

1. Semantic Code Analysis
2. Customizable Security Queries
3. Detection of Logic Flaws and Vulnerabilities
4. Integration with GitHub for continuous scanning
5. Identifies common vulnerabilities (e.g., SQL Injection, XSS)

Git Guardian Shield

1. Secrets Detection (API Keys, Passwords, etc.)
2. Real-Time Commit Scanning
3. Prevents Hardcoded Secrets in Code Repositories
4. Alerts for Sensitive Information Leaks
5. Git Integration for DevOps Security



CodeQL
Language



GitGuardian

SAST limitations

- High False Positives
- Limited to Code at Rest

SAST strengths

- Early Detection in SDLC
- Code-Level Insights

Challenges with SAST

- Complex Configuration
- Scalability for Large Projects

DAST Tools

- Dynamic Application Security Testing (DAST) is a procedure that actively investigates running applications with penetration tests to detect possible security vulnerabilities.
- DAST tools continually search for vulnerabilities in a web application that is in production, hunting for weaknesses that attackers could try to exploit and then illustrating how they could remotely break into the system.
- Upon identifying a vulnerability, a DAST solution sends automated alerts to the appropriate teams so they can prioritize and remediate it.



Zap

- Penetration Testing
- Application Spidering
- Active and Passive Scanning
- Fuzzing Techniques
- Session Hijacking Detection

Nikto

- Web Vulnerability Scanning
- Over 6,700 Known Vulnerability Checks
- Outdated Software Detection
- Customizable Scanning Options
- Compatible with Major Web Servers



Manual limitations

- Time-consuming
- High Expertise Required

Automation Limitations

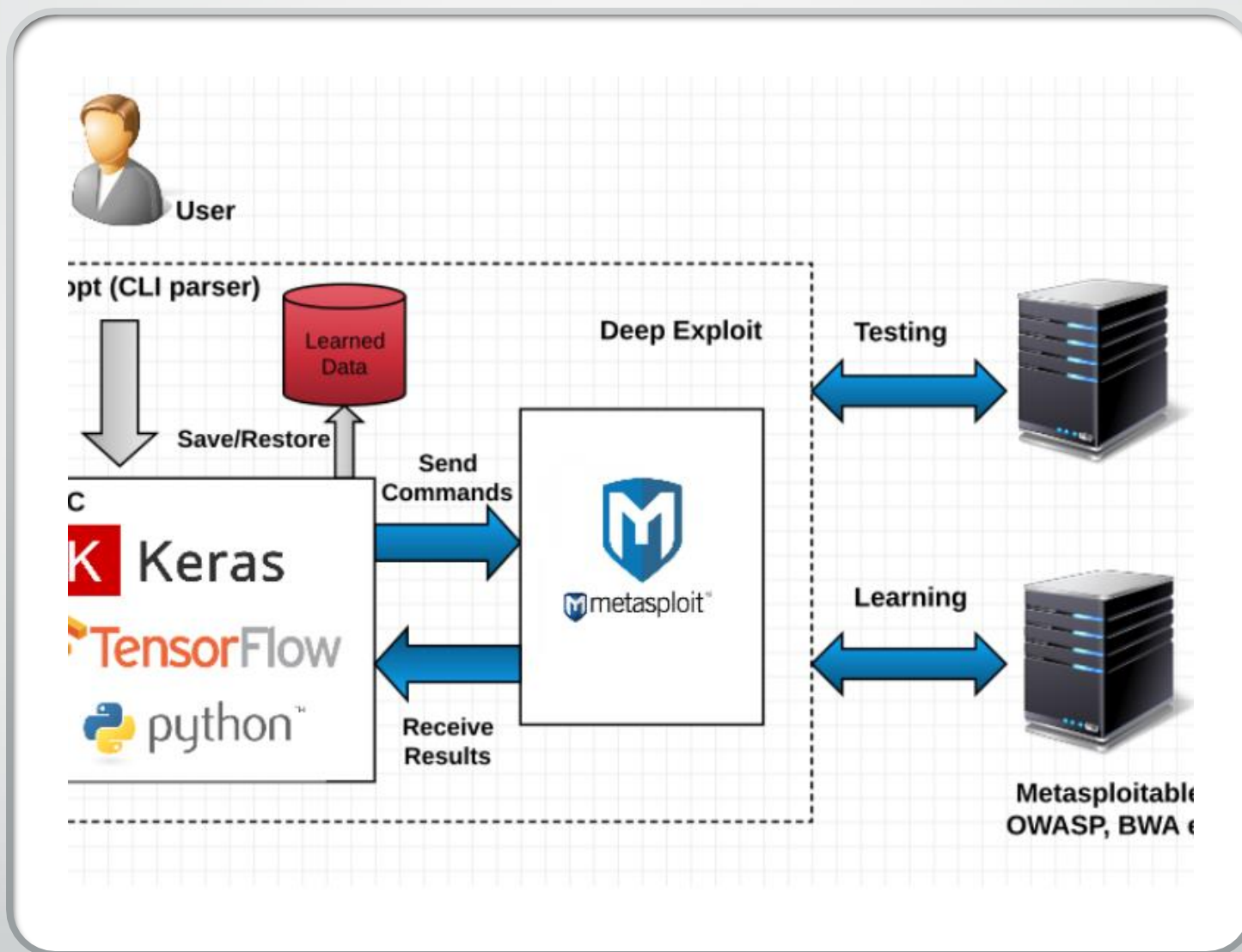
- Predefined vulnerability databases
- High False Positives

AI Integrations

- Vulnerability Pattern Recognition
- Automated Exploit Generation

DEEP EXPLOIT

- It is fully automated penetration testing tool using Deep Reinforcement Learning.
- It identifies the status of all opened ports on the target server and executes the exploit at pinpoint.
- Efficiently execute exploit
- Deep penetration & self-learning.
- Learning time is very fast.



PROJECT DELIVERABLES



A **detailed final report** outlining research methodologies, key findings, and actionable insights.



Well-structured presentation materials tailored for both academic and industry audiences.



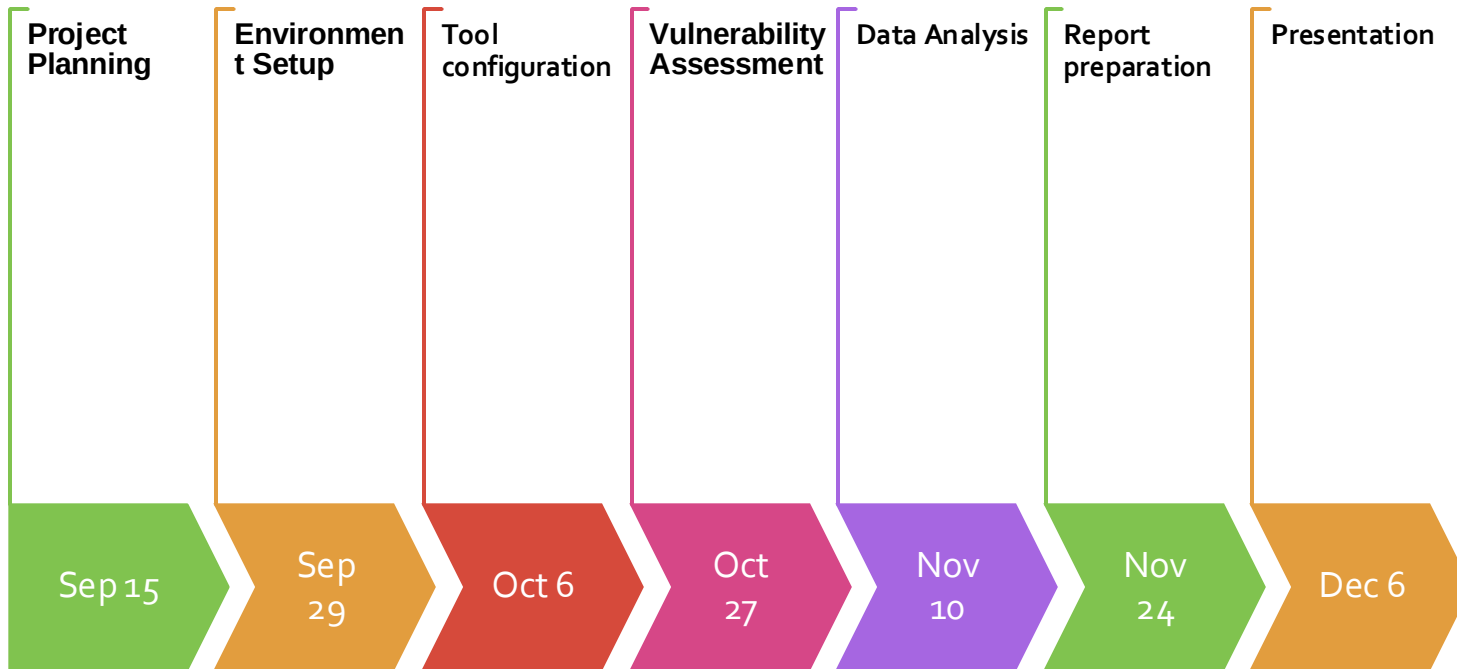
Development of **tool comparison metrics** for evaluating the effectiveness of various SAST and DAST tools.



Recommendations for future work and potential enhancements in integrating AI into vulnerability assessment tools.



Resources and guidelines to enable scholars, developers, and cybersecurity professionals to validate and extend project outcomes.



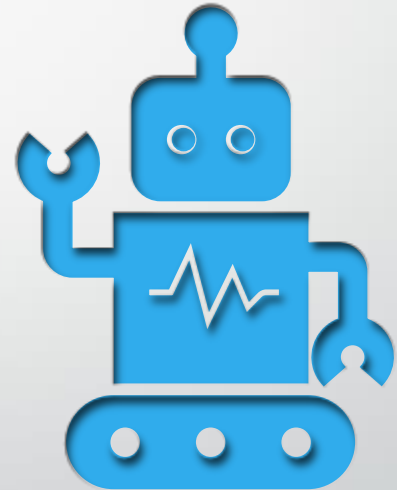
TIMELINE

CONCLUSIONS TO DATE

- Completed an in-depth analysis of various SAST and DAST tools, identifying key strengths and weaknesses in both automated and manual approaches.
- Concluded that automated tools offer scalability and speed, while manual approaches provide deeper contextual understanding in vulnerability detection.
- Discovered that there are very few open-source tools with robust AI integration.
- Identified areas where AI integration could significantly improve vulnerability detection and reduce false positives.
- Highlighted potential gaps in current open-source tools, presenting opportunities for further innovation and research.

ISSUES/ CHALLENGES

- **Tool Selection and Compatibility:** Identifying SAST and DAST tools that align with the project's scope and technologies can be challenging, particularly when aiming for complete and thorough coverage.
- **Resource Demands for Manual Testing:** Manual testing is resource-intensive, requiring specialized expertise and significant time, which may hinder the overall testing timeline.
- **Configuration Challenges:** Automated tools necessitate precise configuration to ensure accurate results and thorough security coverage across all system components.
- **Expertise Dependency:** The effectiveness of manual testing depends heavily on the tester's expertise, making it vulnerable to human error and resulting in potential inconsistencies.



BIBLIOGRAPHY

- NIST, NIST Special Publication 800-53 Revision 5: Security and Privacy Controls for Information Systems and Organizations. National Institute of Standards and Technology, Sept. 2020.
- OWASP. (2023, February). OWASP Top Ten 2023. OWASP Foundation.
- CISA. (2022, July 14). Mitigating Log4Shell and Other Log4j-Related Vulnerabilities. CISA.
- Alazab, M., Tang, M., & Choo, K.-K. R. (2019). Artificial Intelligence in Cybersecurity: A Bibliometric Analysis of the Current State of Research. IEEE Access, 7, 101759–101772.
- Veracode. (2022, November). The State of Software Security: Securing Software Supply Chains. Veracode.



ACRONYMS

- **DAST:** Dynamic Application Security Testing – A method of testing the security of an application by examining it in its running state for vulnerabilities.
- **RCE:** Remote Code Execution – A security vulnerability that allows an attacker to run arbitrary code on a remote machine.
- **SAST:** Static Application Security Testing – A method of analyzing source code or compiled applications for vulnerabilities without executing the code.
- **SDLC:** Software Development Life Cycle – A process for planning, developing, testing, and deploying software systems, typically involving stages like requirements, design, coding, testing, and maintenance.
- **XSS:** Cross-Site Scripting – A type of security vulnerability where an attacker injects malicious scripts into web pages viewed by others.

